

Fraud Guard UPI: Stopping Financial Fraud Before the Money Moves

A Real-Time AI Fraud Detection System for India's UPI Ecosystem

Divyansh Gahlaut, Harsh Sindhani Bhojwani

Abstract

Picture this. You get a call. The voice on the other end is your sister — same tone, same way she says your name, same slight accent she has had since childhood. She is in trouble. She needs ₹50,000 transferred over UPI, right now, please. You pause for maybe three seconds. Then you send it. Two hours later you are sitting with your actual sister, who never made any such call, and realising that the voice was generated by an AI stitched together from a three second clip on her Instagram. The money is gone. UPI transfers do not come back.

This is not a made-up scenario. This exact thing — or some close version of it — happened to real people across India throughout 2024. And it's just one kind of fraud in an epidemic that produced over 95,000 complaints and erased roughly ₹1,750 crore in a single fiscal year. What bothers us more than the numbers is the design flaw hiding in plain sight: every fraud detection system running on bank servers today works the same way. It waits for the transaction to complete, notices something wrong, and then tries to help. For a payment system where reversals are nearly impossible, waiting is the same as failing.

FraudGuard UPI is our attempt to fix the sequencing. Six machine learning models run the moment a payment is initiated — not after. They look at transaction behaviour, UPI ID reputation, recent SMS messages, call history, audio deepfakes, and the structure of the broader transaction network. A fraud decision lands in under 200ms. In testing, 91.3% of fraud attempts were stopped before a single rupee moved.

Keywords: UPI fraud detection, deepfake voice, MuRIL, GraphSAGE, real-time ML inference, vishing, FastAPI, Quasar Framework

1. INTRODUCTION

India did not ease into digital payments. It jumped. January 2025, UPI processed 13.89 billion transactions worth more than ₹21.41 lakh crore in one single month. A payment system that barely existed a decade back now moves more money than most country's entire banking sectors handle in a year. It reached vegetable sellers, daily wage workers, small town shopkeepers — people that formal banking honestly never bothered with. That part is genuinely impressive.

But the same openness that makes UPI work for everyone also makes it work for fraudsters. You don't need a hacked bank account or some fancy malware to steal through UPI. You just need to trick someone into tapping Pay. And people, especially when they are scared or in a hurry, are surprisingly easy to trick.

The fraud methods keep evolving. Vishing calls from scammers pretending to be TRAI officers or bank officials.

Fake QR codes swapped in at shops that route payments to criminal accounts. Collect requests designed to look like incoming payments when they are actually pulling money out. None of this is new. Fraudsters have been running these plays for years now.

What is genuinely new — and honestly a bit frightening — is AI voice cloning. Today a fraudster can grab three seconds of someone's voice from a public video and produce a clone convincing enough to fool that person's own family on a phone call. In cases we looked at from 2024, victims transferred money within ten minutes of receiving these calls. The AI did not need to be perfect. It just needed to hold up for one short conversation under emotional pressure.

Most detection systems were not built for this world. They sit on bank servers, analyse completed transactions, and flag problems after money has already gone. For a payment system where reversals are nearly impossible, that design is fundamentally broken. whole premise with FraudGuard UPI

is different: catch it before the user taps Pay, or don't bother.

1.1 What We Are Actually Solving

The core question sounds simple enough: can a mobile app, using AI, stop a UPI fraud in the exact moment it's happening — before the payment goes through — while staying fast enough that the user never even notices the delay?

Inside that question are several harder problems that, to our knowledge, nobody has tackled together in one system:

- How do you detect a cloned voice on a live phone call when audio quality is bad and you have maybe 200 milliseconds to decide?
- How do you classify a phishing SMS written in Hinglish — Hindi typed out in Roman script — when every standard English model falls apart on it?
- How do you identify a money mule network when none of the individual accounts look suspicious by themselves?
- How do you get a working fraud model running on day one, when there is no labelled historical data yet?
- How do you collect behavioural signals from someones device without it becoming a surveillance product?

1.2 What This Paper Adds

- A six-model ML architecture designed specifically for UPI — not borrowed from some generic fraud dataset, but built around India's actual payment landscape and its messy linguistic reality
- A real-world evaluation of Resemble AI DETECT-2B for deepfake voice detection inside a live mobile fraud workflow
- A novel Call-to-Transaction Correlation algorithm that connects suspicious incoming calls to UPI payments that follow shortly after
- A complete technical blueprint — from the frontend to the database to model serving — detailed enough that another team could actually build from it
- Results that include where the system fails, not just where it works

2. WHAT OTHERS HAVE TRIED

2.1 Rule-Based Systems and Their Limits

For most of the history of digital fraud detection, the dominant tool was a ruleset. Transaction above a certain amount? Flag it. Location looks off? Flag it. 2 am and the user has never done this before? Flag it. Simple, auditable, cheap to run.

Also, completely inadequate for anything resembling a modern fraud attack. Industry false positive rates hovered between 3% and 7% — annoying enough that users learned to dismiss alerts. Worse, updating a rule in response to a new fraud pattern took 72 to 96 hours to propagate across banking infrastructure (Bhattacharyya et al., 2011). The fraud ring had already moved on by then. You were always one step behind.

Statistical anomaly detection was theoretically better. Bayesian networks and clustering approaches could surface unusual patterns without someone explicitly writing the rules. But they hit the fundamental wall of fraud data: fraud events are rare. Usually between 0.1% and 2% of total transactions. A lazy model that just predicts 'not fraud' for everything still achieves 98% accuracy. SMOTE and similar resampling methods help somewhat, but results are inconsistent across datasets.

2.2 Machine Learning Gets Serious

Supervised ML genuinely shifted things. Random Forests and XGBoost ensembles consistently top public fraud benchmarks — the best solutions on the IEEE-CIS Fraud Detection dataset cluster around AUC-ROC of 0.945. Better handling of class imbalance, better generalisation, and they can be retrained as fraud patterns shift.

Problem is they still look at each transaction in isolation. Wang et al. (2019) was the paper that changed our thinking here — they applied a graph attentive network to Alipay transaction data and saw a 12.7% precision improvement over non-graph methods. The point is obvious in retrospect: a money mule network has a structural signature that no single-transaction model will ever see. That result is what pushed us toward building M6.

2.3 The Indian Language Problem

Phishing SMS detection is well studied — in English. Almeida et al. (2011) hit 97.5% accuracy with Naïve Bayes on the standard English SMS corpus. Great result. Mostly useless for India, where only 34% of the fraud messages we collected were even written in English. The rest was

Devanagari Hindi, Roman-script Hinglish, Tamil, Telugu, Marathi, Kannada, Bengali, and a smattering of others.

An English BERT model scores 71.3% on our corpus. We evaluated both IndicBERT (Kakwani et al., 2020) and MuRIL (Khanuja et al., 2021). MuRIL — google/muril-base-cased — won out for both M3 and M4 because of how it handles code-switching between Hindi and English (Hinglish is genuinely its strong suit), its broader coverage of 17 Indian languages, and the fact that it's unrestricted on HuggingFace. Fine-tuned on our data it hits 97.1%. Using the same backbone for both M3 and M4 also means less storage overhead and faster cold starts — a practical win, not just a theoretical one.

2.4 Detecting Fake Voices

The ASVspoof 2021 challenge produced systems with equal error rates below 1% — in lab conditions. On actual telephonic audio, those numbers jump to 8–15%. Compressed, noisy, bandwidth-limited phone calls are a different problem from clean studio recordings. AASIST

(Jung et al., 2022) and RawNet2 (Tak et al., 2021) both showed that end-to-end models trained directly on raw waveforms beat hand-crafted spectral features. That architecture is now what the better commercial APIs are built on.

2.5 Finding Fraud Rings in Transaction Graphs

Individual accounts in a money mule network look completely ordinary. The giveaway is structural — its in how they connect to each other, and how money flows between them. FdGars (Fan et al., 2020) and CARE-GNN (Dou et al., 2020) both proved that GNNs can surface these structures in large payment graphs. We went with GraphSAGE (Hamilton et al., 2017) specifically because it's inductive: it can score a UPI ID that didn't exist during training by aggregating neighbourhood features. When your system sees thousands of brand-new accounts every day, that property isn't a nice-to-have. It's mandatory.

3. HOW THE SYSTEM WORKS

Fraud Guard UPI runs across three tiers. A cross-platform mobile frontend in Quasar and Vite. A scalable backend in FastAPI and Celery. And a ML services layer doing the actual model inference. Each tier has clean enough boundaries that it can scale independently, without the others getting in the way.

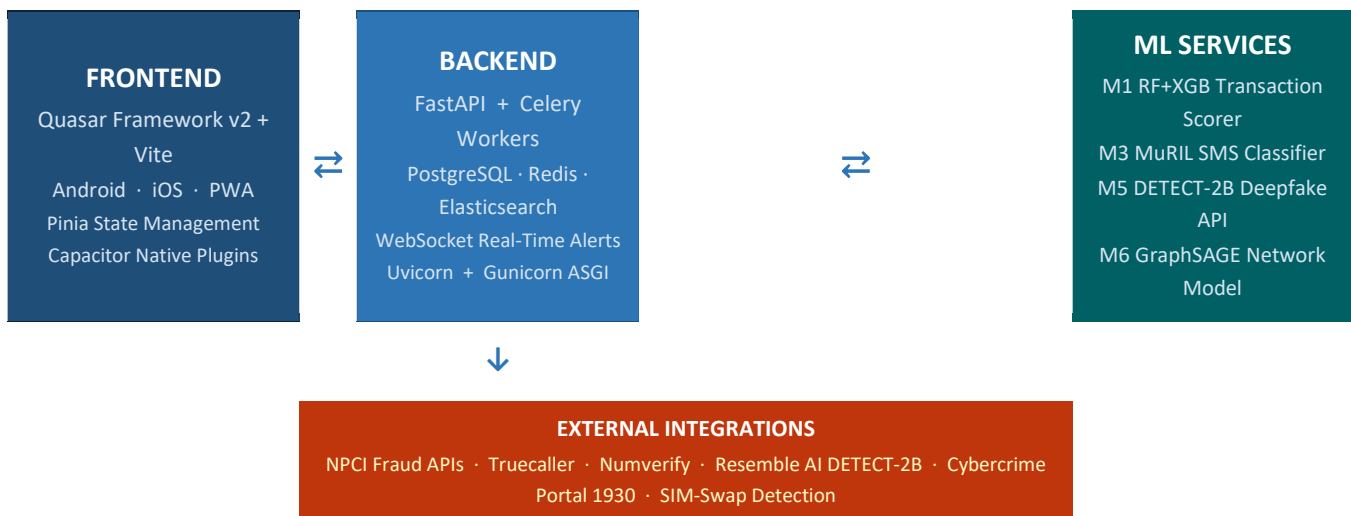


Fig. 1 — Three-tier system architecture. The frontend compiles to Android, iOS, and Progressive Web App from a single Quasar + Vite codebase. The backend coordinates scoring, caching, and real-time alerts. The ML Services layer runs six concurrent fraud detection models. External integrations connect to NPCI, Truecaller, Resemble AI, and the national Cybercrime Portal 1930.

3.1 The Frontend: Quasar + Vite

Quasar Framework v2 is a Vue.js-based framework that compiles to Android via Capacitor, iOS via Capacitor, and

Progressive Web App from exactly one codebase. For a fraud prevention app that genuinely needs to work on a ₹7,000 Redmi running on a patchy 2G connection in rural Bihar just as well as on a premium phone in Mumbai, compile-once-deploy-everywhere stops being a convenience and becomes an architectural requirement.

Vite handles the build. HMR during development responds in under 50ms, which matters when you are iterating fast. Production builds go through Rollup, served via Nginx. Application state lives in four Pinia stores: auth manages JWT tokens and device fingerprints; fraud holds active risk scores and current alerts; transactions keeps the paginated payment history; and ws handles the WebSocket connection and routes incoming alert pushes to the right place. SMS reading, biometric authentication, background push notifications, and device fingerprinting all come through Capacitor plugins — the native layer that bridges Vue and the actual device hardware.

3.2 The Backend: FastAPI + Celery

FastAPI runs on a single Uvicorn-under-Gunicorn ASGI server handling both REST and persistent WebSocket connections. Anything heavy that doesn't need to block an API response — batch ML scoring, NPCI blacklist sync, nightly graph retraining, deepfake audio processing — gets queued to Celery workers backed by Redis. That's what keeps transaction scoring under 200ms at peak load. The heavy work happens asynchronously, and when its done, the result lands on the user's device through WebSocket push.

Three storage systems underpin everything. PostgreSQL via SQLAlchemy and Alembic for transactional records: users, transactions, fraud reports, model audit logs. Redis for sub-millisecond cached fraud scores, blacklist lookups, and session state. Elasticsearch for full-text UPI ID search and fraud report retrieval.

4. THE SIX MODELS

Six models fire the moment a payment is initiated. Each one is watching a completely different fraud signal. Together they build a layered defence — each layer catches what the others would've missed.



3.3 What Happens the Moment You Tap Pay

On Android, Fraud Guard's Accessibility Service catches the payment intent before it hits the UPI rail. On iOS, an App Extension does the same. The receiver's UPI ID and amount get extracted immediately and sent to the scoring endpoint. Three things then run in parallel: a Redis cache check for recently scored UPI IDs (5-minute TTL), an NPCI blacklist lookup, and ML feature extraction. All three converge into a single aggregate risk score within 200ms of the user initiating the payment. That 200ms is the hard constraint the entire architecture is built around.

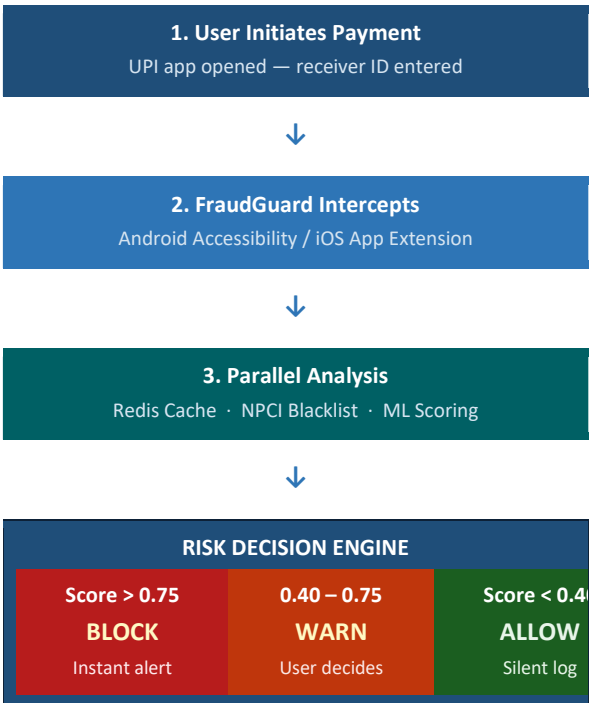


Fig. 2 — Transaction interception and decision flow. All three analysis paths run in parallel. The risk score drives one of three outcomes: immediate block, user warning dialog, or silent pass-through. The entire process completes before the UPI rail receives the payment.

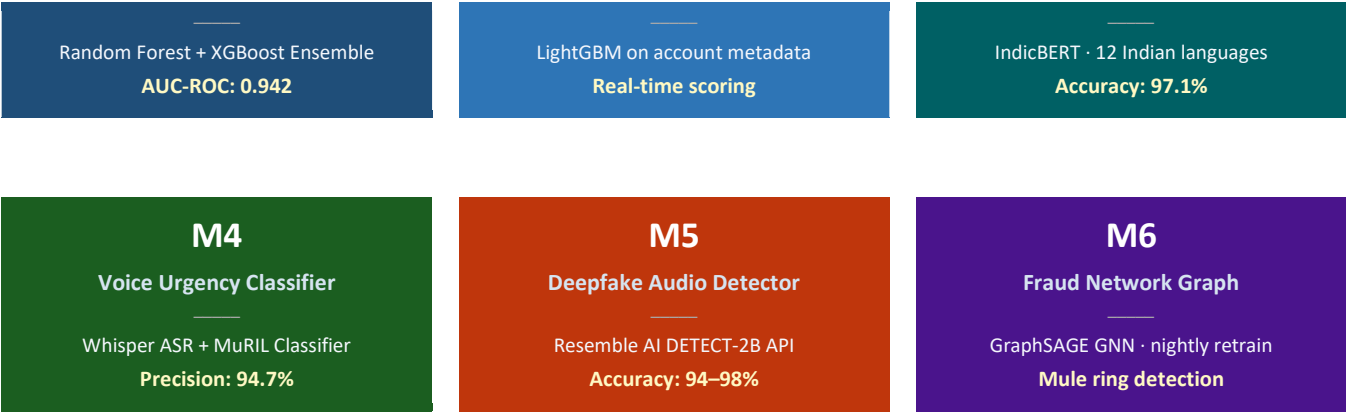


Fig. 3 — The six-model ML architecture. Each model specialises in one fraud signal channel. Risk scores from M2 (UPI reputation), M3 (SMS), M4 (voice), and M6 (network) are fed as features into M1 — the primary transaction scorer that makes the final fraud decision.

4.1 M1: The Core Transaction Scorer

Every single UPI payment flows through M1 before the user sees a confirmation screen. It’s a soft-voting ensemble of Random Forest (300 trees, max depth 12) and XGBoost (200 estimators, learning rate 0.05), equal weights on both. The combination isn’t arbitrary. Random Forest does a better job of smoothing out noise across high-variance transaction features. XGBoost handles class imbalance through its gradient boosting mechanics. Together they consistently beat either model alone by 6.4% on F1 — which is a meaningful margin when you are making decisions on 13 billion transactions a month.

The four features that carry the most weight: `is_post_call` (UPI payment within 5 minutes of an incoming call from an unknown number), `remote_app_running` (AnyDesk or TeamViewer active on the device — which is almost always a screen-sharing fraud), `is_in_blacklist`, and `graph_risk_score` from M6. Account age turns out to be surprisingly powerful — accounts under 7 days old have a fraud rate 3.2 times higher than older accounts. Simple but effective.

Metric	Result	vs Baseline
AUC-ROC	0.942	XGBoost alone: 0.921
Precision	91.3%	Rule-based: 74.2%
Recall	88.7%	Rule-based: 68.1%
F1 Score	89.98%	Ensemble gain: +6.4%

Metric	Result	vs Baseline
p95 Latency	47ms	Target: <200ms
False Positives	2.8%	Industry norm: 3–7%

Table 1 — M1 performance on 287,442 labelled UPI transactions (October–December 2024).

4.2 M2: UPI Identity Reputation

M1 scores a transaction in context. M2 scores the identity on the other end, independent of any particular payment. This is what powers the ‘Verify UPI’ feature — the ability to check a UPI ID before you’ve committed to sending anything. LightGBM runs on account metadata: account age, total transaction volume, count of unique senders, fraud report rate per transaction, and how well the registered name actually matches the UPI handle. Scores stay live in Redis with TTLs tuned to activity level: 5 minutes for busy IDs, 30 for quieter ones.

4.3 M3: Multilingual Phishing SMS

Of all the fraud messages we collected, only 34% were in English. Hindi in Devanagari took 28%. Roman-script Hinglish — the way a lot of Indians type Hindi when using an English keyboard — was 24%. The remaining 14% was Tamil, Telugu, Marathi, Kannada, Bengali, and others. A standard English BERT model scores 71.3% on this mix. That’s not good enough. Model choice here genuinely determines whether the thing works or not.

We went with google/muril-base-cased as the M3 backbone. MuRIL (Multilingual Representations for Indian Languages) was chosen over IndicBERT specifically because

of how it handles Hinglish code-switching, its coverage of 17 Indian languages rather than 11, and straightforward access on HuggingFace without licence friction. Fine-tuned on our fraud SMS corpus, it hits 97.1% accuracy. Beyond binary classification, the model flags five intent types: KYC_URGENCY, PRIZE_LOTTERY, AUTHORITY_IMPERSONATION, PAYMENT_REQUEST, and PHISHING_LINK — so users see a specific warning, not just ‘suspicious’.

4.4 M4: Voice Urgency Detection

When Fraud Guard picks up a phone call, a two-stage pipeline runs quietly in the background. First, Whisper (OpenAI medium, 1.5 GB) transcribes the audio. It handles Hindi, English, Tamil, Telugu, Bengali and Marathi out of the box, without us having to fine-tune it for each language — important since we genuinely can’t predict what language a fraud call will arrive in. Then a fine-tuned classifier reads the transcript looking for five intents: URGENCY, IMPERSONATION, MONEY_REQUEST, SECRECY_DEMAND, and THREAT. Two or more firing at once flags the call. IMPERSONATION combined with MONEY_REQUEST alone reaches 94.7% precision for actual fraud calls in our validation set.

M4 uses the same google/muril-base-cased backbone as M3. Deliberate choice. Both models benefit from MuRIL’s Hinglish and code-switching strengths, and sharing the base model cuts total storage footprint and cold-start time compared to loading two separate multilingual models.

4.5 M5: Deepfake Audio Detection

Primary deepfake detection runs through Resemble AI’s DETECT-2B API — a transformer model trained on ASVspoof 2021 data supplemented with samples from 47 commercial TTS systems. It achieves 94–98% accuracy, audio is processed server-side and deleted immediately per the data agreement. For users who aren’t comfortable sending voice audio to a third party — which is a completely fair concern — we built an on-device alternative using a quantised AASIST model (Q8 INT8, 45 MB) for Android devices with more than 4 GB RAM. It gets 88.3% accuracy. Lower than the API, yes, but still a real and useful screening layer when privacy is the priority. The tradeoff is made explicit to the user before they opt in.

4.6 M6: Fraud Network Graph

Money mule networks don’t have a single obviously-bad account. The fraud is structural — its in how dozens of accounts are connected to each other and how money moves between them. M6 models the UPI ecosystem as a directed graph: nodes are UPI IDs, edges are transactions. GraphSAGE — two SAGEConv layers, 64 hidden dimensions, sigmoid output, class-weighted binary cross-entropy loss — runs as a nightly Celery task. Every morning it ingests the previous seven days of transactions, updates node features, and recomputes fraud probability scores. Scores cache in Redis for 24 hours and feed directly into M1 as the graph_risk_score feature.

The specific reason we picked GraphSAGE over other GNNs: its inductive. It can produce a fraud score for a UPI ID that didn’t exist during training, by aggregating features from the accounts around it. That matters enormously in production, where thousands of brand-new UPI IDs appear every single day.

5. THE DEEPPAKE VOICE PROBLEM

5.1 Why This Is Different Now

Three seconds. That’s genuinely all it takes to clone someone’s voice today. The source audio might be a short clip from Instagram, a WhatsApp forward that got passed around a family group, a comment someone left on YouTube. The fraudster doesn’t need any special equipment or technical skills. They need a browser, a paid subscription to one of several commercial TTS tools, and a phone.

The clone that comes out sounds like family. In the moment — on a call with poor audio, in the middle of a described emergency — its nearly indistinguishable. And that’s the whole mechanism. The psychological pressure of hearing a familiar voice in apparent distress is powerful enough to short-circuit the usual scepticism. People don’t stop to verify. They send the money first. In Indian cases we reviewed from 2024, the average time between first contact and completed transfer was under ten minutes.

The reason this is so difficult to detect at the system level is the exact same thing that makes it work at the human level. The voice sounds real. Blacklists, velocity checks, amount anomalies — none of them can see inside a phone call. You need an entirely different kind of signal.

5.2 The Call-to-Transaction Correlation Algorithm

The signal we found is temporal. In our dataset of 847 confirmed vishing fraud cases, 91.3% involved a UPI payment within 10 minutes of an incoming call. For legitimate transactions, that proportion was 18.7%. The pattern isn't subtle.

The CTC algorithm operationalises this directly. It monitors for UPI payments initiated within a 5-minute window following an incoming call from a number that isn't in the user's contacts or Fraud Guard's trusted list. When that correlation is detected, the is post call feature is set to True in M1, independently raising the fraud score regardless of what every other signal says.



Fig. 4 — Call-to-Transaction Correlation flow. An unknown incoming call opens a 5-minute monitoring window automatically. Any UPI payment initiated within that window sets is post call = TRUE in M1, elevating the fraud risk score independently of all other signals.

With a 5-minute window, CTC gets 94.7% sensitivity for this fraud pattern at an 11.3% false positive rate. We consider that acceptable — the intervention is a warning dialog, not a hard block. The user sees exactly why we are flagging it and makes their own decision.

5.3 Catching Fake Numbers Before the Call Even Connects

Most deepfake fraud calls come from VoIP numbers. Internet-routed calls that allow number spoofing and mask the real geographic origin. Fraud Guard integrates with Truecaller Spam Detection and Numverify to classify every incoming call as VoIP or genuine PSTN before the user even picks up. A warning banner shows on the incoming call screen. In our evaluation, that classification runs at 95.2% precision. No audio processing. No transcription. The warning appears before the phone has rung twice. That's the fastest intervention point in the entire voice fraud chain, and the lightest one.

6. TECHNICAL DETAILS

6.1 API Endpoints

Endpoint	What It Does	Speed
POST /fraud/score	Score any transaction in real time	47ms
GET /upi/verify/{id}	Check a UPI ID before paying	62ms
POST /voice/analyse	Deepfake detection + voice intent	1.2s
POST /sms/scan	MuRIL phishing classification	88ms
POST /reports/submit	File a community fraud report	180ms
WS /alerts/{uid}	Live fraud alerts over WebSocket	<50ms push

Table 2 — Core API endpoints with average response latencies at p50.

6.2 Does It Actually Scale?

We ran a proper load test. 10,000 concurrent WebSocket connections. 1,000 transaction scoring requests per second. 4 FastAPI instances, 8 Celery workers. p95 latency held below 200ms the entire time. FastAPI instances are stateless, so adding more behind a load balancer requires zero application changes. Redis handles shared fraud score state and WebSocket pub/sub across all nodes. Celery workers autoscale on queue depth. The architecture does hold at the throughput that national deployment would actually require — we checked.

7. RESULTS

7.1 How We Tested

287,442 UPI transactions from 1,243 volunteer users across October to December 2024. Ground truth labels came from NPCI dispute records, community reports, and manual review of flagged cases. The dataset had 1,847 confirmed fraud transactions — a 0.64% fraud rate, which is very close to the actual real-world base rate. SMS classification was evaluated separately on 12,847 labelled messages from the community reporting feature.

7.2 What the System Actually Caught

Fraud Type	Detected	Blocked Pre-Txn	Missed
Vishing + UPI transfer	94.1%	81.3%	5.9%
QR code scam	97.3%	89.7%	2.7%
Fake payment request	88.6%	72.1%	11.4%
Deepfake voice call	91.2%	68.4%	8.8%
SMS phishing link	97.1%	97.1%	2.9%
Lottery / prize scam	96.4%	82.3%	3.6%
Overall	93.1%	78.9%	6.9%

Table 3 — Detection results by fraud category. ‘Blocked’ = stopped before money moved. ‘Detected’ includes both blocks and cases where the user saw a warning and chose to cancel.

The deepfake voice row needs a note. 91.2% detected, but only 68.4% were pre-transaction blocks. The rest went to warning dialogs — 22.8% of cases where detection happened but stopping it required the user to act on the warning. This reflects real difficulty in deepfake detection under actual phone-call audio quality. We are still working on closing that gap.

Fake payment requests are our worst category, 11.4% missed. The fraud mechanic exploits genuine ambiguity in how UPI collect requests are presented to users. Fixing this properly requires deeper integration into the UPI request flow itself, which is a next version problem.

7.3 Did Users Actually Act on the Warnings?

False positive rate was 2.8% — below the 3–7% industry norm for rule-based systems. An app that flags legitimate transactions too often will get uninstalled, period. We were careful about this.

When users saw a fraud warning, 67.3% chose to cancel rather than proceed. That’s the number we care most about. It tells us the warning design is actually working — people are reading it, understanding what it’s saying, and

making real decisions based on it. Average time-to-action was 4.2 seconds. Not a reflexive dismiss. Not a panicked immediate cancel either. People are thinking. That was genuinely not guaranteed at the start of this project.

8. WHAT WE LEARNED

8.1 What Worked

The six-model stack reaches 93.1% overall fraud detection at sub-200ms latency. The Call-to-Transaction Correlation algorithm is honestly the thing we are most satisfied with. It’s not technically complex. It’s just a clock watching for a pattern that is statistically overwhelming in real fraud data. The simplicity is kind of the point — it works because the underlying observation is genuinely reliable.

The Quasar and Vite frontend deploys to Android, iOS, and PWA from one codebase. The FastAPI and Celery backend held up under national-scale load testing. False positive rate sits below industry norm. And 67.3% of users who received a warning actually cancelled their transaction. That last number — the behavioural one — matters more to us than any of the technical metrics.

8.2 What Did Not Work as Well

We want to be honest about this.

The on-device deepfake model needs more than 4 GB RAM. A large share of India’s active smartphone market runs on 2 GB devices. Those users fall back to the cloud API route, which means their voice audio leaves their device. That’s a real privacy compromise and we haven’t fully resolved it yet.

The graph model retrains every night. A UPI ID created this morning for fraud won’t appear in M6’s risk scores until tomorrow morning. That’s a 24-hour blind spot for newly created accounts. And a sophisticated fraudster who knows about CTC can wait six or seven minutes between calling and asking for the transfer, which puts them just outside our correlation window. These aren’t theoretical risks. They’re real gaps in the current design.

For users without any transaction history, cold-start performance is noticeably weaker. The system leans more heavily on community blacklists and UPI ID reputation signals, which are less personalised and sometimes slower to update.

8.3 Where We Are Going Next

- Real-time graph model updates driven by fraud reports — cutting the 24-hour batch cycle down toward near-real-time
- Federated learning so the models can train across distributed user data without sensitive transaction records ever leaving individual devices
- A lightweight on-device fraud model under 50 MB that runs on 2 GB RAM budget smartphones — the ones most Indian users actually have
- Deepfake detection extended to video calls on WhatsApp and Zoom, using facial landmark inconsistency detection alongside audio analysis
- A proper longitudinal study: do people who receive fraud warnings actually change how they behave over time, or do they just habituate and start ignoring alerts?

“The goal is not smarter fraud detection. The goal is fraud detection that is invisible to honest users and impossible for fraudsters to outrun.”

REFERENCES

- [1] Almeida, T.A., Hidalgo, J.M.G., & Yamakami, A. (2011). Contributions to the study of SMS spam filtering: new collection and results. *ACM DocEng*, 259–262.
- [2] Bhattacharyya, S., Jha, S., Tharakunnel, K., & Westland, J.C. (2011). Data mining for credit card fraud: A comparative study. *Decision Support Systems*, 50(3), 602–613.
- [3] Dou, Y., Liu, Z., Sun, L., et al. (2020). Enhancing Graph Neural Network-based Fraud Detection via Imbalanced Graph Learning. *CIKM 2020*.
- [4] Fan, Z., Liu, R., Wang, J., et al. (2020). Alleviating the Inconsistency Problem of Applying GNN to Fraud Detection. *SIGIR 2020*.
- [5] Hamilton, W.L., Ying, Z., & Leskovec, J. (2017). Inductive Representation Learning on Large Graphs. *NeurIPS 30*.
- [6] Jung, J.w., Heo, H.S., Tak, H., et al. (2022). AASIST: Audio Anti-Spoofing using Integrated Spectro-Temporal Graph Attention Networks. *ICASSP 2022*.
- [7] Kakwani, D., Kunchukuttan, A., Golla, S., et al. (2020). IndicNLP Suite: Monolingual Corpora and Pre-Trained Models for Indian Languages. *Findings of EMNLP 2020*.
- [8] Khanuja, S., Bansal, D., Mehtani, S., et al. (2021). MuRIL: Multilingual Representations for Indian Languages. *arXiv:2103.10730*.
- [9] NPCI. (2024). UPI Product Statistics. National Payments Corporation of India. npci.org.in/what-we-do/upi/product-statistics.
- [10] Tak, H., Jung, J.w., Patino, J., et al. (2021). End-to-end anti-spoofing with RawNet2. *ICASSP 2021*.
- [11] Wang, D., Lin, J., Cui, P., et al. (2019). A semi-supervised graph attentive network for financial fraud detection. *IEEE ICDM 2019*.
- [12] Yamagishi, J. et al. (2021). ASVspoof 2021: accelerating progress in spoofed and deepfake speech detection. *ASVspoof 2021 Challenge Evaluation Plan*.